MODULE 17 · DOMAIN 4 - MONITORING AND LOGGING

Collecting Logs and Metrics

The CloudWatch agent, custom metrics and the embedded metric format, log groups and log classes, routing logs out in real time, and centralising across accounts.

DOMAIN 4 of 6 EXAM WEIGHT 15% MODULE 17 of 34

admin@sthithapragna.com

No dashboard can show a signal that was never **captured**. Collection has four steps: a **source**, a **collector**, a **store**, and a **route** onwards.

Where signals come from

SOURCE	CAPTURED HOW	NEEDS YOU TO ACT?
AWS service metrics	Published to CloudWatch automatically	No
Service logs: VPC Flow, ALB, CloudTrail	Vended logs, once enabled	Enable them
Operating system: memory, disk space	CloudWatch agent	Install and configure
Application logs and metrics	Agent, SDK, or embedded metric format	Instrument
Distributed traces	X-Ray or OpenTelemetry	Instrument (module 19)

ON THE EXAM

EC2 memory and free disk space are not default metrics. That single fact answers a whole family of questions.

EC2 metrics: basic, detailed, and missing

- **Basic monitoring**: CPU, network and disk operations every 5 minutes, free.
- **Detailed monitoring**: the same metrics every minute, charged.
- Neither includes **memory** or **disk space used**. Those need the CloudWatch agent.
- Status check metrics report instance and system health, as in module 14's recovery alarms.

> ON THE EXAM
>
> Faster detection of CPU spikes: detailed monitoring. Any memory or disk-space alarm: the agent.

The CloudWatch agent

- Collects system metrics, process metrics, StatsD and collectd metrics, and log files.
- Runs on EC2 and on-premises servers, Linux and Windows.
- Store its configuration in **Parameter Store**; install and update it with Systems Manager across the fleet.
- Needs permission to publish: the CloudWatchAgentServerPolicy on the instance role.

> ON THE EXAM
>
> Hundreds of instances, consistent configuration, least effort: config in Parameter Store, deployed with Systems Manager.

An agent configuration

```
{ "metrics": {
    "append_dimensions": { "AutoScalingGroupName": "${aws:AutoScalingGroupName}" },
    "metrics_collected": {
      "mem":  { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] } } },
  "logs": { "logs_collected": { "files": { "collect_list": [
      { "file_path": "/var/log/app/*.log",
        "log_group_name": "/app/orders",
        "retention_in_days": 30 } ] } } } }
```

2 Tag every metric with its group, so you can alarm per fleet.

9 Set retention here, or log groups keep data forever.

ON THE EXAM

Aggregate by Auto Scaling group to alarm on a fleet, not on instances that come and go.

Four ways to publish custom metrics

METHOD	HOW	BEST WHEN
PutMetricData API	Your code calls CloudWatch	Low volume, any environment
CloudWatch agent	StatsD or collectd from the app	Servers already running the agent
Embedded metric format	Structured JSON logs become metrics	Lambda and containers at high volume
Metric filter	A pattern extracts values from logs	Logs already contain the number

ON THE EXAM

No code change allowed and the value is already logged: a metric filter. High-volume Lambda: EMF.

Metric essentials

- A metric is a **namespace**, a **name** and up to 30 **dimensions**; each combination is a separate metric.
- Standard resolution is one minute; **high resolution** goes down to one second.
- High-resolution alarms can evaluate every 10 or 30 seconds.
- A dimension such as customer ID creates one metric per customer: cardinality becomes cost.

> ON THE EXAM
>
> Need per-request detail? Put it in logs and query it. Keep metric dimensions low in cardinality.

The embedded metric format

```
1 { "_aws": { "Timestamp": 1758528000000,
2     "CloudWatchMetrics": [{ "Namespace": "Orders",
3       "Dimensions": [["Service", "PaymentMethod"]],
4       "Metrics": [{ "Name": "OrderValue", "Unit": "None" }] }] },
5   "Service": "checkout",
6   "PaymentMethod": "card",
7   "OrderValue": 84.50,
8   "OrderId": "o-7731",
9   "CustomerTier": "gold" }
```

2-4 Instructions: which fields become a metric, and its dimensions.

8-9 Extra fields stay in the log line, queryable, without becoming metrics.

ON THE EXAM

One log line is both a metric and a searchable record. No API calls, no added latency, no throttling.

Log groups: retention, encryption, protection

- Default retention is **never expire**. Set retention on every log group, ideally in infrastructure as code.
- Encrypt with a **KMS key** per log group; the key policy must allow the Logs service.
- **Data protection policies** detect and mask sensitive data, such as card numbers, as logs arrive.
- Masked values can be seen only by principals with the logs:Unmask permission.

> ON THE EXAM
>
> Sensitive data appears in logs and code cannot change quickly: a data protection policy masks it on ingestion.

Standard or Infrequent Access log class?

INFREQUENT ACCESS

Cheaper to ingest

- Lower price per GB ingested
- Querying, encryption, cross-account analysis
- No real-time features such as filters
- Chosen when the log group is created

STANDARD

Full features

- Metric filters and subscription filters
- Live Tail and anomaly detection
- Embedded metric format extraction
- The default

THE TIEBREAKER

Logs kept for audits and occasional forensics: Infrequent Access. Logs driving alarms or pipelines: Standard.

Getting logs out of a log group

ROUTE	TIMING	NOTES
Export task to S3	Batch	One task at a time per account
Subscription filter	Real time	Lambda, Kinesis, Firehose; two per group
Account-level filter	Real time	One per account; includes new groups
Cross-account destination	Real time	To a stream in another account
Centralisation rules	Continuous copy	Organisation-wide, cross-Region (2025)

ON THE EXAM

'Real time' rules out export tasks. 'Every log group, including new ones' points to the account-level filter.

A real-time log pipeline

01

Log group

Application and service logs arrive.

02

Subscription filter

Matches events, account-wide if needed.

03

Firehose

Buffers, and a Lambda can transform.

04

Destination

S3, OpenSearch, or a third-party tool.

05

S3 lifecycle

Cheaper tiers, then expiry.

ON THE EXAM

Firehose is the managed default for delivery. Kinesis Data Streams adds custom consumers and replay.

Cross-account observability, or log centralisation?

LOG CENTRALISATION

Copy the logs

- Copies across accounts and Regions
- Organisation, OUs or chosen accounts
- Adds @aws.account and @aws.region fields
- First copy free: compliance retention

CROSS-ACCOUNT OBSERVABILITY

Look without copying

- Monitoring account sees source accounts
- Metrics, logs and traces, via sink and links
- No copy; data stays in the source
- Within a Region

THE TIEBREAKER

Retain everything centrally for years, across Regions: centralisation. One pane of glass for operators: observability.

Metric streams

- Stream CloudWatch metrics continuously through **Firehose**, to S3 or a partner tool.
- Filter by namespace and metric; choose JSON or OpenTelemetry format.
- Metric streams carry **metrics only**. Logs use subscription filters.
- Metrics can also be centralised across accounts and Regions with centralisation rules.

> ON THE EXAM
>
> Send metrics to a third-party monitoring tool in near real time: a metric stream, not API polling.

Service logs worth knowing

LOG	DELIVERED TO	ANSWERS
VPC Flow Logs	CloudWatch Logs, S3, Firehose	Which traffic was accepted or rejected
ALB access logs	S3	Every request: client, latency, status
CloudTrail	S3, and optionally CloudWatch Logs	Who called which API
S3 server access logs	S3	Requests made to a bucket
API Gateway access logs	CloudWatch Logs, Firehose	Every API request
Lambda logs	CloudWatch Logs; JSON and level controls	Function output and errors

ON THE EXAM

Load balancer access logs go to S3 only. Query them with Athena, not CloudWatch Logs Insights.

Collecting from containers

- The **awslogs** driver sends container output to CloudWatch Logs.
- **FireLens** runs Fluent Bit beside the app, routing logs to many destinations with filtering.
- **Container Insights** collects cluster, service, task and pod metrics for ECS and EKS.
- The awslogs driver needs log permissions on the execution role, or the container instance role on EC2.

> ON THE EXAM
>
> Send container logs to several destinations with filtering: FireLens. Simple CloudWatch delivery: awslogs.

A company runs 300 Linux EC2 instances across several Auto Scaling groups. Applications have crashed when instances ran out of memory, with no warning.

The operations team needs alarms when memory utilisation exceeds 85% for any group, applied consistently to every instance with the least operational overhead.

What should the DevOps engineer do?

A Enable detailed monitoring on every instance

B Add a cron job on each instance that calls PutMetricData with free memory

C Store a CloudWatch agent configuration collecting memory in Parameter Store, and deploy the agent with Systems Manager

D Enable AWS Compute Optimizer for the instances

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Detailed monitoring

More frequent CPU and network metrics. Still no memory.

B Cron and PutMetricData

Works, but custom scripts on 300 instances, and new instances need them too.

C Agent via Systems Manager

The agent collects memory; one stored config and SSM deployment keep every instance consistent.

D Compute Optimizer

Makes sizing recommendations. It does not alarm.

THE SIGNAL

Memory or disk space: the agent. At scale: configuration in Parameter Store, deployment with Systems Manager.

> A high-volume checkout function on Lambda must publish an order-value metric with dimensions for service and payment method. Individual order IDs must remain searchable.
>
> An earlier attempt using PutMetricData added latency and hit API throttling during sales.

Which approach should the developers use?

A Write structured logs in the embedded metric format

B Batch PutMetricData calls at the end of each invocation

C Create a metric filter on the existing unstructured logs

D Record order values as X-Ray annotations

◆ PAUSE AND DECIDE

Which approach should the developers use?

A Embedded metric format

Metrics extracted asynchronously from log lines: no API calls, no throttling, order IDs stay queryable.

B Batched PutMetricData

Fewer calls, but still API calls in the request path, still throttled at scale.

C Metric filter on unstructured logs

Fragile pattern-matching, and awkward for multiple dimensions.

D X-Ray annotations

Annotations index traces for searching. They are not CloudWatch metrics.

THE SIGNAL

High volume, dimensions, no API calls, records still searchable: the embedded metric format.

> An audit finds that an application occasionally writes full card numbers into CloudWatch Logs. A code fix will take several weeks.
>
> Until then, card numbers must be masked in the logs as they arrive, and only the security team may view unmasked values.

What should the DevOps engineer do?

A Create a metric filter that counts card numbers

B Apply a data protection policy with the card number identifier, and grant only the security team logs:Unmask

C Add a subscription filter that sends logs to a Lambda function which removes card numbers

D Encrypt the log group with a customer managed KMS key

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Metric filter

Counts occurrences. The card numbers remain visible.

B Data protection policy with Unmask

Masks matching data as it arrives; only principals with logs:Unmask see the original.

C Scrubbing Lambda

Cleans a copy elsewhere. The original log group still holds the card numbers.

D Customer managed key

Anyone allowed to read the logs still sees the numbers. Encryption is not masking.

THE SIGNAL

Mask sensitive data in logs without code changes, with selective viewing: a data protection policy.

> A company with 40 accounts in AWS Organizations, operating in three Regions, must keep a copy of all application logs in a central log archive account for seven years.
>
> It wants the least operational overhead and no custom code.

Which solution meets these requirements?

A. CloudWatch cross-account observability links from every account to the log archive account

B. CloudWatch Logs centralisation rules scoped to the organisation, copying to the log archive account

C. Subscription filters in every account and Region, streaming to Kinesis in the log archive account

D. Nightly export tasks from every log group to an S3 bucket in the log archive account

◆ PAUSE AND DECIDE

Which solution meets these requirements?

A **Cross-account observability**

Views data in place without copying, and within one Region. No retained copy.

B **Centralisation rules**

Copies across accounts and Regions, organisation-wide, with source fields added. No code.

C **Subscription filters to Kinesis**

The pre-2025 design: 120 set-ups, streams and consumers to run.

D **Nightly exports**

Batch jobs to schedule across every log group, with gaps and custom automation.

THE SIGNAL

A retained copy, many accounts, many Regions, no code: centralisation rules.

> An operations team works in a monitoring account. Thirty application accounts run in the same Region.
>
> Operators want to search logs, view metrics and follow traces across all thirty accounts from the monitoring account, without copying any data.

What should the DevOps engineer set up?

A. Log centralisation rules to the monitoring account

B. Metric streams from each account to the monitoring account

C. A shared CloudWatch dashboard in each application account

D. CloudWatch cross-account observability: a sink in the monitoring account and links from the application accounts

◆ PAUSE AND DECIDE

What should the DevOps engineer set up?

A **Centralisation**

Copies logs, which is ruled out, and does not cover metrics and traces together.

B **Metric streams**

Metrics only, and a copy.

C **Shared dashboards**

Thirty separate dashboards, and no searching or tracing across accounts.

D **Cross-account observability**

Logs, metrics and traces viewed in place from one account, in the same Region.

THE SIGNAL

Pair this with scenario 4: copy for retention, link for viewing.

> A team must investigate slow requests through an Application Load Balancer: client IP, target, response time and status code for every request.
>
> It wants to run ad hoc SQL queries over months of data at the lowest cost.

What should the DevOps engineer do?

A Enable VPC Flow Logs on the load balancer's subnets

B Create alarms on the load balancer's TargetResponseTime metric

C Enable ALB access logs to S3, and query them with Athena

D Enable CloudTrail data events for Elastic Load Balancing

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A VPC Flow Logs

Network flows, not HTTP requests. No response times or status codes.

B Response time alarms

Aggregated metrics tell you something is slow, not which requests.

C ALB access logs and Athena

Every request, with all four fields, stored cheaply in S3, queried with SQL.

D CloudTrail data events

Records API calls to the ELB service, not traffic through the load balancer.

THE SIGNAL

Per-request HTTP detail from a load balancer: access logs in S3, and Athena to query them.

Module 17 in five lines

- Source, collector, store, route. EC2 memory and disk space need the CloudWatch agent.
- Custom metrics: API, agent, embedded metric format, or metric filters. Watch cardinality.
- Set log retention; mask sensitive data with data protection policies; IA class for rare queries.
- Real time out: subscription filters, account-level for every group. Export tasks are batch.
- Centralisation copies across accounts and Regions; cross-account observability views in place.

ON THE EXAM

Next, module 18: analysing and alarming. Logs Insights, anomaly detection, composite alarms and Synthetics.